

AI HR RECRUITMENT ASSISTANT

IBM INTERNSHIP — AGENTIC AI FINAL PROJECT REPORT

Domain: Agentic Artificial Intelligence / Recruitment Automation

Application: AI-Powered HR Recruitment & Candidate Screening

Prepared by: Sudharsana B

Core Architecture: Agent + Tools + RAG

1. Project Overview

1.1 Project Title

HireMate AI – Agentic AI-Powered HR Recruitment & Candidate Management System

1.2 Problem Statement

Traditional recruitment operations often depend on manual resume screening, repetitive candidate comparisons, static job descriptions, interview coordination, and human effort for routine recruitment tasks. This can increase hiring time, create inconsistent screening decisions, increase recruiter workload, and delay identification of suitable candidates.

HireMate AI addresses this problem by providing an intelligent first-line recruitment support layer that understands recruiter requests, retrieves relevant recruitment policies and job requirements through RAG, screens and matches candidate information using tools, and supports interview preparation and candidate management.

1.3 Brief Description of the Project

HireMate AI is a functional Agentic AI recruitment application built using Python, LangChain, LangGraph, Ollama, Qwen 2.5 3B, ChromaDB, Nomic Embed Text, PyPDF, JSON, and Streamlit. The system combines four major capabilities: an AI agent for decision making, RAG for knowledge-grounded recruitment policies and job guidance, tools for candidate and recruitment actions, and memory for conversational continuity.

1.4 System Components

Component	Role
AI Agent	Understands recruiter requests and selects the appropriate recruitment action.
RAG	Retrieves job requirements, recruitment policies, interview guidance, and HR knowledge from the knowledge base.
Tools	Perform resume screening, candidate lookup, job matching, interview question generation, and recruitment actions.
Memory	Maintains conversational context during a recruiter session.
Streamlit UI	Provides the dashboard and AI HR Recruitment chat interface.

2. Objectives & Proposed Solution

2.1 Project Objectives

- Provide intelligent automated first-line HR recruitment support.
- Retrieve relevant job requirements, recruitment policies, and interview guidance from an HR knowledge base.
- Use an AI agent to decide which recruitment action is appropriate.
- Automate resume screening and candidate-to-job matching.
- Generate structured interview questions based on job requirements.
- Maintain conversational context using memory.
- Provide candidate profile and application status information when required.
- Support professional and user-friendly recruitment workflows.

2.2 How the Agentic AI Solution Works

The recruiter submits an HR request through the Streamlit interface. The LangGraph-based agent, powered by Qwen 2.5 3B through Ollama, interprets the request and determines whether knowledge retrieval or a recruitment tool is required.

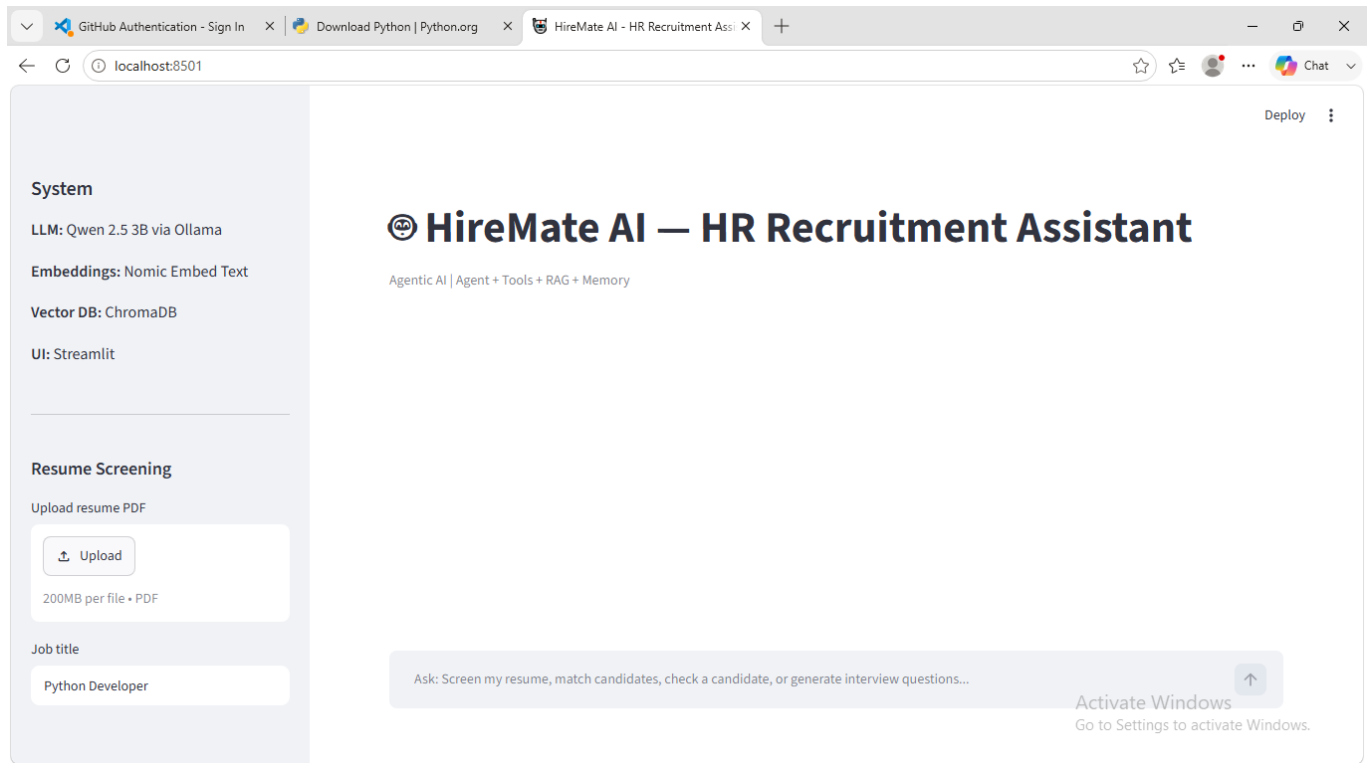
Step	Process	Output
1	Recruiter submits a recruitment inquiry	Recruiter request
2	Agent interprets the request	Intent / action decision
3	Job/policy issue → RAG search; operational request → tool call	Relevant knowledge / candidate profile / action result
4	Memory maintains session context	Context-aware response
5	Tool execution such as screening or candidate matching	Recruitment action result
6	Agent generates final response	Recommendation or action result

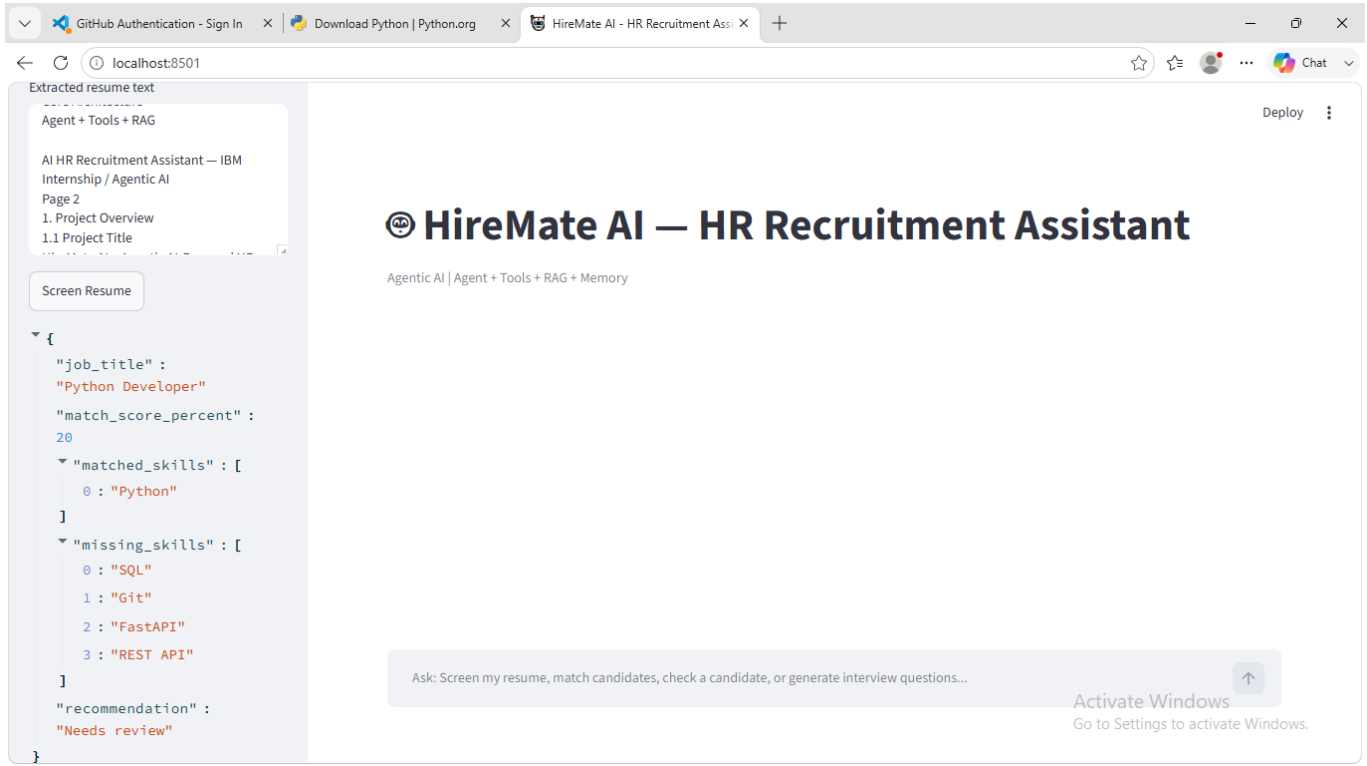
2.3 Key Features

Feature	Description
RAG Knowledge Search	Searches recruitment policies, job descriptions, interview guidelines, and HR knowledge documents.
Resume Screening	Extracts relevant candidate skills, education, experience, and qualifications.
Candidate Matching	Matches candidates with job descriptions and identifies suitable roles.
Interview Questions	Generates role-specific interview questions based on job requirements and candidate profiles.

Candidate Information	Retrieves simulated candidate ID, skills, experience, application status, and profile information.
Recruitment Status	Checks and updates candidate application or interview status.
Conversation Memory	Uses session memory to understand recruiter follow-up questions.
Dashboard	Displays system status, candidate metrics, recruitment activity, and capabilities.

3. Implementation & Results





3.1 Technologies / Tools Used

Technology/Tool	Purpose
Python	Core application development.
LangChain	LLM, embedding, prompt, and tool integration.
LangGraph	Agent workflow, state management, and tool routing.
Ollama	Local language model execution.
Qwen 2.5 3B	Language model used by the agent.
ChromaDB	Vector database for RAG.
Nomic Embed Text	Document embedding model.
PyPDF	PDF loading and processing.
Streamlit	Web-based dashboard and chat interface.
JSON	Candidate, application, and recruitment data storage.

3.2 Working Process

Five HR knowledge-base PDF documents were prepared for recruitment policies, job descriptions, resume screening guidelines, interview practices, and candidate evaluation. The ingestion pipeline loads the PDFs with PyPDFLoader, splits the content with RecursiveCharacterTextSplitter, generates embeddings using Nomic Embed Text, and stores the vectors in ChromaDB.

The agent is connected to the RAG retriever and operational recruitment tools. The LangGraph workflow contains an agent node and ToolNode with conditional tool routing. After a tool returns a result, the agent generates the final response. Candidate and application records are stored in JSON. Session-based conversational memory is implemented using LangGraph MemorySaver and a thread ID.

3.3 Tools and Functions

Tool Function	Purpose
search_knowledge_base	Searches the HR knowledge base using similarity search.
screen_resume	Screens a candidate resume against job requirements.
match_candidate	Matches candidates with suitable job descriptions.
get_candidate_information	Returns simulated candidate profile and application information.
generate_interview_questions	Generates interview questions for a selected role.
update_application_status	Updates the status of a candidate application.
schedule_interview	Creates a simulated interview scheduling action.

3.4 Screenshots / Output

The working HireMate AI application can demonstrate the dashboard, AI HR Recruitment interface, RAG-based policy response, resume screening result, candidate matching, interview question generation, application records, and memory-based follow-up interaction.

- Dashboard with recruitment metrics and system status
- Resume screening and candidate score output
- Job-to-candidate matching result
- Interview question generation
- Candidate application status

3.5 Results Achieved

Test Input	Request	Result
TC-01	Screen candidate resume for Software Developer	Resume skills and experience were compared with the job requirements.
TC-02	Find the best candidate for a Python Developer role	Candidate matching tool identified suitable candidates.
TC-03	What is the status of CAND-1001?	Candidate profile and current application status were retrieved.
TC-04	Generate interview questions for CAND-1001	Role-specific interview questions were generated.
TC-05	Show recruitment policy for interview screening	RAG retrieved relevant HR policy and screening guidance.
TC-06	Update CAND-1001 application to Shortlisted	Application status was updated successfully.

TC-07	I already reviewed this candidate. What should I check next?	Previous conversation context was used for the next response.
-------	--	---

Overall Result

The implemented system demonstrates how Agentic AI can support recruitment workflows by combining grounded knowledge retrieval, operational tools, and conversational memory. The agent can assist recruiters with repetitive screening and candidate-management activities while keeping responses tied to the available HR knowledge base.

4. System Architecture

4.1 High-Level Architecture

Recruiter → Streamlit UI → LangGraph Agent → Decision Layer → RAG / Recruitment Tools → Result → Agent Response

Layer	Main Responsibility
User Interface	Recruiter interacts through Streamlit chat and dashboard.
Agent Layer	LangGraph agent interprets intent and decides the next action.
Knowledge Layer	ChromaDB + embeddings provide grounded HR and job knowledge.
Tool Layer	Resume screening, candidate matching, status update, interview generation, and scheduling.
Memory Layer	MemorySaver maintains context across recruiter follow-up requests.
Data Layer	JSON stores simulated candidate and application records.

4.2 Agent Decision Flow

- Receive recruiter request.
- Identify whether the request needs HR knowledge, candidate data, or an operational action.
- Retrieve relevant documents through RAG when policy or job knowledge is required.
- Call the appropriate recruitment tool when an action is required.
- Use conversation memory for follow-up context.
- Return a structured and professional response.

5. Advantages and Applications

5.1 Advantages

- Reduces repetitive manual resume screening effort.
- Provides consistent candidate-to-job matching support.
- Makes HR policy and recruitment guidance easier to retrieve.
- Generates interview questions quickly for different roles.
- Maintains conversational context for recruiter follow-up questions.
- Provides a single interface for multiple recruitment tasks.
- Can be extended with enterprise recruitment platforms and databases.

5.2 Practical Applications

- Initial resume screening for entry-level and experienced roles.
- Candidate shortlisting assistance.
- Job description and skill requirement analysis.
- Interview preparation and question generation.
- Candidate application status support.
- Recruiter knowledge-base assistance.
- Recruitment operations dashboard and reporting.

5.3 Limitations

- The project uses simulated candidate and application records.
- The local language model may require additional validation for production recruitment decisions.
- The current implementation does not replace human judgement in final hiring decisions.

6. Conclusion & Future Scope

6.1 Project Conclusion

HireMate AI demonstrates a practical application of Agentic AI for HR recruitment automation. The project goes beyond a basic chatbot by combining an AI agent with RAG, operational tools, and conversational memory.

The implemented system can retrieve grounded recruitment guidance, screen resumes, match candidates with job descriptions, generate interview questions, retrieve candidate information, update application status, and maintain session context. The Streamlit interface provides a clear environment for live demonstration.

6.2 Challenges Faced

- Configuring the local Python environment and required AI libraries.
- Handling package compatibility and import paths during development.
- Preparing and ingesting multiple HR and recruitment PDFs into the vector database.
- Connecting the RAG retriever and tools to the LangGraph agent workflow.
- Preventing repeated tool execution and maintaining controlled agent routing.

- Maintaining conversational context across follow-up recruiter requests.
- Designing a professional dashboard while keeping the system suitable for local execution.

6.3 Future Enhancements

Persistent database-backed candidate memory.

- Integration with enterprise ATS platforms and recruitment systems.
- Email and SMS notification support.
- User authentication and role-based access control.
- Real-time job-board and candidate-profile integration.
- Advanced resume parsing and skill extraction.
- Administrator dashboard with analytics and recruitment reporting.
- Bias and fairness monitoring for recruitment workflows.
- Voice-based recruiter support.
- Cloud deployment and scalable infrastructure.

6.4 References

- LangChain documentation — LLM, RAG, embeddings, and tool integration.
- LangGraph documentation — agent workflows, state management, tools, and memory.
- Ollama documentation — local language model execution.
- Chroma documentation — vector database and similarity search.
- Streamlit documentation — Python-based web application interface.

7. Appendix — Demonstration Flow

Step	Action	Capability
1	Open Dashboard	Professional UI, system status, candidate metrics
2	Upload / review a resume	Resume Screening Tool
3	Ask for candidate-job matching	Agent + RAG + Matching Tool
4	Check candidate status	Candidate Information Tool
5	Generate interview questions	Interview Question Tool
6	Update application status	Application Update Tool
7	Ask a follow-up question	Conversation Memory

Project Summary

AI HR Recruitment Assistant applies Agentic AI concepts to a practical recruitment workflow. The combination of Agent + Tools + RAG enables the system to interpret recruiter requests, retrieve grounded knowledge, perform recruitment actions, and maintain conversational continuity.